

Disease Predictor Module – Detailed Report Document

1. Module Title

AI-Based Potato Leaf Disease Predictor with Severity Assessment, Visual Explanations, and Smart Recommendations

2. Abstract

This module detects potato leaf diseases from images and provides actionable agronomic guidance. It supports two input modes: manual image upload and live capture through ESP32-CAM. A trained deep learning classifier predicts one of three classes: Early Blight, Late Blight, or Healthy. The module then performs OpenCV-based lesion analysis to estimate diseased area percentage and generates explainable visual outputs. Severity is computed from diseased area percentage using fixed thresholds: Critical (80–100%), High (50–79%), Moderate (20–49%), Low (0–19%). The frontend intentionally hides confidence values and emphasizes practical severity and recommendations for decision-making.

3. Problem Statement

- Potato farmers require early, field-ready diagnosis of leaf diseases.
- Manual diagnosis is slow and depends heavily on expert availability.
- Existing systems often provide only class labels without severity context or treatment-level recommendations.
- This module addresses that gap by combining classification + lesion quantification + recommendation support in a single interface.

4. Objectives

- Detect major potato leaf disease classes from images.
- Quantify visible disease spread on leaf surface.
- Convert quantified disease spread into a clear severity level.
- Provide immediate treatment/fertilizer guidance.
- Enable real-time field use through ESP32-CAM integration.
- Present outputs in a farmer-friendly UI without exposing technical confidence metrics.

5. Scope of the Module

In Scope

- Leaf disease classification (3 classes).
- Disease-area computation and severity mapping.
- Visualization outputs for interpretability.
- Rule-based and AI-generated recommendations.

- Web-based frontend integration with backend REST APIs.

Out of Scope

- Multi-leaf batch inference with geostatistical analytics.
- Offline model retraining pipeline inside the production app.
- Full farm-level temporal forecasting.

6. System Architecture (Module View)

- Input Layer: Uploaded leaf image (JPG/PNG/WEBP) or ESP32-CAM captured image.
- Inference Layer: Image preprocessing -> CNN model inference -> class probabilities and predicted class.
- Analysis Layer: OpenCV lesion segmentation and leaf masking -> diseased area percentage.
- Recommendation Layer: Built-in recommendations + Gemini AI recommendations.
- Presentation Layer: Predicted class, severity gauge, disease-area visuals, class probability chart, treatment guidance.

7. Data Flow / Workflow

1. User submits image (upload) or triggers ESP32 capture.
2. Backend validates and stores temporary image.
3. Image is resized and normalized for model inference.
4. CNN predicts disease class.
5. OpenCV extracts leaf region and lesion region.
6. Diseased area percentage is computed.
7. Severity is derived from diseased area thresholds.
8. Recommendations are generated and returned to frontend.
9. Frontend renders severity-focused results (without confidence display).

8. Core Technical Design

8.1 Input Handling

- Upload API accepts image files in multipart form.
- ESP32 API pulls image from device capture endpoint and runs the same prediction pipeline.
- Temporary files are cleaned after processing.

8.2 Preprocessing for Model

- Image read and color conversion.
- Resize to 224 × 224.
- Pixel normalization to [0, 1].

- Tensor shape prepared for single-image inference.

8.3 Classification

- Model output is a softmax probability vector.
- Predicted class = $\text{argmax}(\text{probabilities})$.
- Supported labels: Early Blight, Late Blight, Healthy.

8.4 Disease Area Estimation (OpenCV)

- Leaf region estimated in HSV color space (green mask + morphology).
- Lesion candidates estimated from dark/necrotic spots, yellow/chlorotic areas, and gray dead tissue.
- Lesion mask constrained to leaf mask and cleaned morphologically.
- Visual outputs: leaf area with contours, disease mask, and combined overlay.

8.5 Disease Area Formula

Disease Area % = $(\text{Disease Pixels} / \text{Leaf Pixels}) \times 100$

8.6 Severity Mapping (Current Requirement)

- Critical: 80–100%
- High: 50–79%
- Moderate: 20–49%
- Low: 0–19%

8.7 Recommendation Engine

- Rule-based baseline recommendations per disease class.
- AI recommendation endpoint for personalized agronomic advice.

9. API Specification (Module-Relevant)

- POST /api/predict-disease
- Input: image file.
- Output: predicted class, class probabilities, recommendation object, visualization object (including disease_area_pct).
- POST /api/predict-from-esp32
- Input: ESP32 IP.
- Output: same structure as predict-disease with source metadata.
- POST /api/ai-recommendation
- Input: disease context and area metrics.
- Output: structured AI treatment/fertilizer recommendations.

- GET /api/health
- Output: backend/model readiness status.

10. Frontend UX Design Decisions

- Two operation modes: Upload mode and ESP32 live mode.
- Result screen prioritizes detected disease, severity level, disease-area visualization, and actionable recommendations.
- Confidence values are not shown in disease predictor UI.

11. Error Handling and Robustness

- Input validation for missing/unsupported files.
- Graceful API error messages in UI.
- ESP32 timeout handling and user guidance.
- Temporary file cleanup in backend finally blocks.
- Fallback logic for AI model availability.

12. Testing and Verification Approach

- Backend health endpoint validation.
- End-to-end manual tests for upload, ESP32 capture, visual outputs, and severity category thresholds.
- Frontend build verification.

13. Key Achievements

- End-to-end disease diagnosis pipeline is functional.
- Severity is converted to practical field-level categories.
- Explainability improved via visual overlays and masks.
- IoT integration supports real-time field capture.
- Recommendation layer links AI output to farmer action.

14. Limitations

- Accuracy depends on image quality and lighting.
- Lesion segmentation is heuristic and may vary under extreme conditions.
- Three-class scope limits broader disease coverage.
- AI recommendation quality depends on external API availability and quota.

15. Future Improvements

- Add more disease classes and local-language support.
- Add confidence calibration for internal QA.

- Add trend analytics and hotspot monitoring.
- Add image quality checks before inference.
- Add offline/on-device lightweight inference options.

16. Conclusion

The Disease Predictor module delivers a practical precision-agriculture component by combining deep learning classification, image-based severity estimation, and recommendation support. Its major contribution is translating model output into farmer-usable insights through severity categorization and visual explanation. With expanded validation and data coverage, the module can evolve into a robust decision-support tool for potato disease management.